

회귀 모델: 선형회귀와 로지스틱회귀

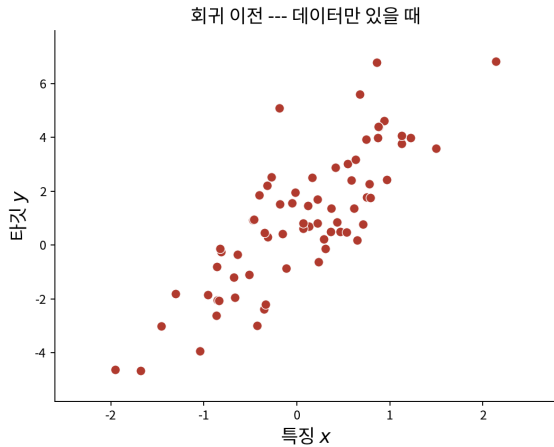
선형회귀: 모델, 비용함수, 경사하강법 · 정규방정식과 “같은 모델, 다른 출력”의 다리 ·
로지스틱회귀: 시그모이드에서 PR-AUC까지

Machine Learning 1 · 2주차

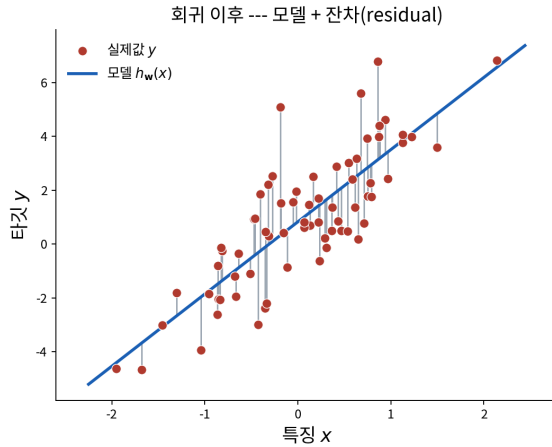
한국과학영재학교 (KSA)

Week 2

선형회귀란?



선형회귀란?



이번 주차 개요

이 주차를 마치면:

- **모델 · 비용함수 · 최적화 알고리즘** 세 부분 뼈대를 세우고, 선형회귀와 로지스틱회귀 양쪽에 구체적으로 조립.
- 평균제곱오차의 그래디언트를 유도하고 경사하강법을 수행한 뒤, **학습률과 조건수**가 수렴·발산에 미치는 영향을 설명.
- 닫힌 형태 해 $w^* = (X^T X)^{-1} X^T y$ 를 유도하고, 경사하강법과의 트레이드오프를 계산 비용과 **사영** 해석으로 짚음.
- 시그모이드 → 확률 → 교차 엔트로피 → 경사하강법 전체 파이프라인을 설명하고, 혼동행렬 · Precision · Recall · F1 · PR-AUC 로 분류기를 평가.

2.1 선형회귀: 모델, 비용함수, 경사하강법

첫 완전한 알고리즘: 세 부분 뼈대

지난 장에서 세운 것은 알고리즘이 아니라 **문제의 정의**였다. 이번 장은 1.2의 예고(“같은 선형모델을 숫자로 읽으면 회귀”)를 **이 학기의 첫 완전한 알고리즘**으로 실행하는 장이다.

알고리즘의 뼈대는 항상 같은 세 부류:

- ① **모델** — 입력을 출력으로 매핑하는 함수의 형태
- ② **비용함수**(loss) — “얼마나 틀렸는지”를 숫자로 재는 재
- ③ **최적화 알고리즘** — 파라미터를 그 숫자가 줄어드는 방향으로 바꾸는 방법

셋이 모여야 비로소 “알고리즘”. 이 틀은 **학기 내내 이름만 바꿔 반복**된다. 그 사이사이에 손으로, 또는 몇 줄의 코드로 재현 가능한 **작은 수치 실험**이 들어간다.

모델: 특징의 가중합

선형회귀는 입력 $\mathbf{x} = (x_1, \dots, x_n)$ 에서 출력을 이렇게 예측한다:

$$h_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n, \quad h_{\mathbf{w}} : \mathbb{R}^n \rightarrow \mathbb{R}$$

- $w_0 =$ **절편**(bias), $w_1, \dots, w_n =$ 각 특징의 **가중치**
- 표기 단순화: $x_0 = 1$ 을 항상 추가하면

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = \sum_{j=0}^n w_j x_j$$

— bias까지 하나의 **내적**으로 통일

파라미터는 어떻게 정해지는가: “맞춤” 작업

$h_w(x) = w^T x$ 에서 우리가 조절할 수 있는 것은 $w = (w_0, \dots, w_n)$ 뿐이다.

학습 알고리즘의 임무

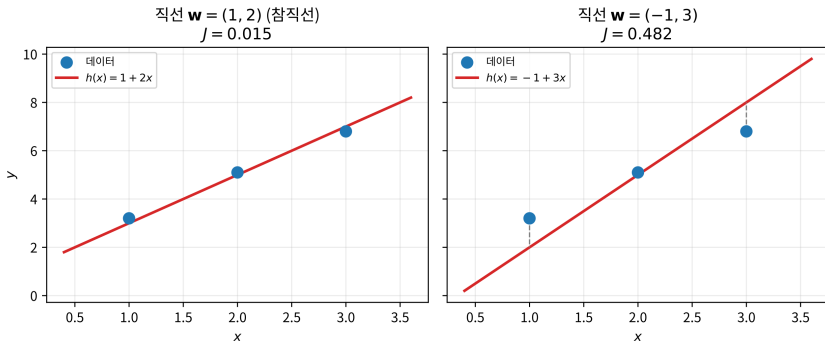
학습 데이터 $(x^{(i)}, y^{(i)})$ 에 대해 예측값 $h_w(x^{(i)})$ 가 실제 출력 $y^{(i)}$ 에 가깝게 되는 w 를 찾는다.

특징이 하나뿐이면 ($n = 1$) $h_w(x) = w_0 + w_1x$ 는 **직선**이다. 임무는 무수히 많은 직선 중 **데이터 점들에 가장 잘 들어맞는 한 줄**을 찾는 **맞춤(fitting)** 작업이 된다.

“잘 들어맞다”를 숫자로

작은 예: 데이터 $(1, 3.2), (2, 5.1), (3, 6.8)$ — 참값 $y = 2x + 1$ 에 소량의 잡음이 섞인 것.

같은 "직선"이지만 오차(점선)의 크기가 다르다



둘 다 “직선”이지만 오차(점선)의 **숫자 크기**만 다르다. **“가장 잘”이라는 말은 데이터가 스스로 알려주지 않는다. “얼마나 틀린지를 어떤 숫자로 재느냐”를 먼저 정의한 다음에야 뜻이 생긴다.** — 비용함수가 부수 주제가 아니라 알고리즘의 진짜 출발점인 이유.

비용함수: 평균제곱오차

m 개의 학습 데이터에 대해, 모델이 얼마나 틀렸는가:

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

왜 오차를 제곱해서 더하는가?

- 오차를 그냥 더하면 양수·음수 오차가 **상쇄** — 엉망인 모델도 “평균 오차 0” 가능
- 절댓값은 상쇄는 막지만 미분이 **불연속** → 최적화가 까다로움
- **제곱은 두 문제를 동시에 해결**: 항상 양수 + 매끄럽게 미분 가능

$\frac{1}{2}$, 그리고 볼록함수

- 앞의 $\frac{1}{2}$ 는 나중에 미분할 때 제곱의 지수 2와 상쇄돼 식이 깔끔해지도록 넣은 관례적 상수. 최솟값의 위치는 안 바뀐.
- $J(w)$ 는 w 에 대해 **이차함수**(quadratic): $w_j w_k$ (2차), w_j (1차), 상수의 합으로만 이루어짐.

볼록함수 판정 — 헤시안 한 계산

이차함수가 볼록함수(convex)인지는 **헤시안**(Hessian, 2차 편미분 행렬)이 양정치인지 한 계산으로 판정:

$$\frac{\partial^2 J}{\partial w \partial w^T} = \frac{1}{m} X^T X$$

— bias 열을 붙인 데이터 행렬 X 의 $X^T X$ 에 비례.

장난감 데이터: J 는 엄밀히 볼록

- 장난감 데이터 ($X = [1, 2, 3]$, bias 열 포함)에서

$$\frac{1}{3} \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2 & 14/3 \end{bmatrix}$$

- 이 행렬의 두 고유값 $\approx 0.120, \approx 5.547$ — **모두 양수** $\rightarrow J$ 는 **엄밀히 볼록**(strictly convex).
- 볼록 이차함수 = 매끄러운 U자 그릇 $\rightarrow$ **지역 최솟값이 없다**. 전역 최솟값이 정확히 하나 있고, 경사하강법은 어디서 출발하든 그 점에 도달한다.

Week 9에서 손실이 비볼록해지면 이 성질이 사라지고, 경사하강법에 보조 장치가 많이 붙는다.

고유값 비율 46:1 — “늘어짐”의 첫 단서

두 번째 고유값과 첫 번째 고유값의 비율:

$$5.547/0.120 \approx 46$$

- 기하학적 뜻: $J(w)$ 의 골짜기 단면이 두 고유방향으로 **약 46:1로 늘어진 타원**.
- 이 “늘어짐”을 2.2절에서 **조건수**(condition number)라고 이름 붙이고, (a) 경사하강법 경로가 얼마나 zigzag하는지, (b) 특징 스케일을 바꾸면 안전한 학습률 범위가 얼마나 달라지는지를 바로 이 숫자로 재게 된다.

조건수가 이 절 전체의 “느림과 발산”을 정량화하는 숫자다.

경사하강법 (Gradient Descent)

$J(w)$ 를 최소화하는 w 를 찾기 위해, 그래디언트(가장 가파르게 증가하는 방향)의 **반대** 방향으로 조금씩 이동:

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}, \quad \frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m \left(h_w(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

- α = **학습률(learning rate)** — 한 걸음의 크기
- 모든 w_j 를 **동시에** 업데이트하는 과정을 손실이 충분히 작아질 때까지 (또는 정해진 반복 횟수만큼) 반복

업데이트는 왜 “동시”인가

모든 w_j 를 “동시” 업데이트해야 한다 — “그냥 그렇게 쓰는 거다”로 넘기면 코드에서 실제로 부딪히는 함정이 된다.

틀린 예: 순차 업데이트

```
w0 -= a * grad0  
w1 -= a * grad(w0, w1)
```

둘째 줄의 grad가 이미 바뀐 w0로 계산됨 — “이동 전”과 “이동 후”가 섞인다.

맞는 예: 동시 업데이트

```
g0, g1 = grad(w0, w1)  
w0 -= a * g0  
w1 -= a * g1
```

grad를 전부 계산한 다음에야 $w[j] -= \dots$ 로 반영 — 두 단계 구조.

순차 업데이트는 더 이상 현재점의 ∇J 가 아니다 — 볼록함수에서 “경사 반대 방향으로 가면 확실히 내려간다”는 보장이 무너진다.

그래디언트는 왜 “오차 × 특징” 형태인가

(2) $\partial J / \partial w_j$ 가 그 합이 되는 이유

각 데이터점 i 는 오차의 제곱을 J 에 기여. 제곱의 미분에 연쇄법칙을 쓰면 2가 나와 $\frac{1}{2}$ 와 상쇄.
 $h_w(\mathbf{x}^{(i)})$ 를 w_j 로 미분하면 $x_j^{(i)}$ **하나**가 남는다. 데이터점 i 의 기여:

$$\left(h_w(\mathbf{x}^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)} \quad \text{— 오차} \times \text{해당 특징의 값}$$

실전 감각으로 번역: **그래디언트는 “데이터점들의 불만 합”**. 오차가 큰 점은 큰 소리로 불평하고, 특징 값이 큰 점은 “더 책임”이 있어 w_j 를 더 크게 조정하라고 불평한다.

2.3절 로지스틱회귀에서 “오차 × 특징” 패턴이 시그모이드를 통과해도 그대로 살아남는 것을 확인 — 그때 유도가 한 줄이 된다.

경사하강법 구현 (순수 Python)

```
def gradient_descent(X, y, alpha, epochs):
    m, n = len(X), len(X[0])
    w = [0.0] * (n + 1) # w[0]=bias
    for _ in range(epochs):
        grad = [0.0] * (n + 1)
        for i in range(m):
            pred = w[0] + sum(w[j+1] * X[i][j] for j in range(n))
            error = pred - y[i]
            grad[0] += error
            for j in range(n):
                grad[j+1] += error * X[i][j]
        for j in range(n + 1):
            w[j] -= alpha * grad[j] / m
    return w
```

“grad 계산 → 반영”의 **두 단계 구조**가 동시 업데이트를 구현한 것. w 는 0에서 시작 — 관례일 뿐, 볼록함수라면 어디서 시작하든 같은 최솟값에 도달.

수렴 경계: 발산하는 α 는 정확히 계산된다

선형회귀는 J 가 이차함수라 **수렴·발산 경계값을 정확히 계산**할 수 있다. 업데이트는 매 스텝 “현재점과 최솟값 w^* 의 오차”에 고정 행렬을 곱하는 선형 연산:

$$w^{(t+1)} - w^* = \left(I - \frac{\alpha}{m} X^T X \right) (w^{(t)} - w^*)$$

(증명은 2.2절 — 여기서는 결과를 받아들이고 숫자를 본다.)

장난감 데이터에서 $\frac{1}{m} X^T X$ 의 고유값이 ≈ 0.120 과 ≈ 5.547 이므로, 매 스텝 오차가 고유방향별로 $(1 - \alpha \cdot 0.120)$ 배, $(1 - \alpha \cdot 5.547)$ 배. 두 방향 모두 줄려면 $|1 - \alpha \cdot 5.547| < 1$, 즉

$$0 < \alpha < \frac{2}{5.547} \approx 0.361 = \alpha^*$$

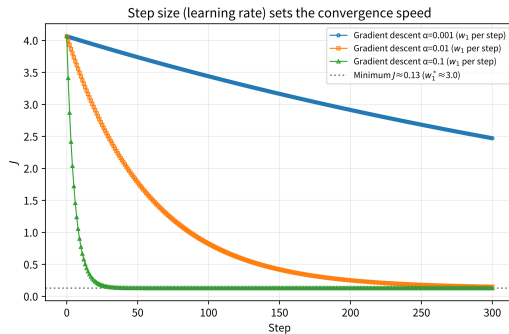
$\alpha^* \approx 0.361$ 이 운명을 정한다

학습률 α	α/α^*	운명
0.1	0.28배	수렴 (조금 느리게)
0.5	1.38배	발산 — 1.5보다 “조금 작은” 값이지만
1.5	4.2배	심각하게 발산

놀라운 결과: $\alpha = 0.5$ 도 발산한다. “발산한 1.5보다만 작으면 된다”는 직감이 틀린 것이다.

안전한 범위는 **데이터** — 정확히 말해 $(1/m)X^T X$ 의 고유값 — 가 전부 결정한다. 이것이 2.2절의 특징 스케일링이 “경사하강법의 걸음 수를 바꾸는 핵심 전처리”인 이유의 정량적 근거다.

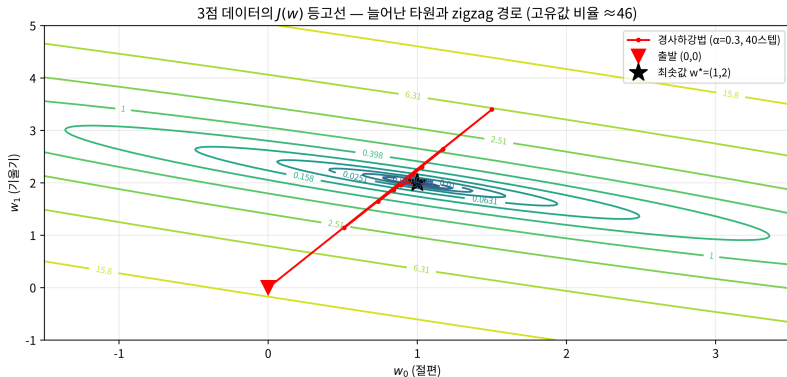
1D 단면: 걸음 크기가 속도를 정한다



$w_0 = 1$ 고정, 세 학습률($\alpha = 0.1, 0.01, 0.001$)로 w_1 만
경사하강 — 스텝별 비용 J , 같은 출발점.

- 세 곡선은 **같은 점에서 출발해 같은 최솟값** (점선)으로 수렴.
- $\alpha = 0.1$ 이 가장 빠르게 바닥에 닿고,
 $\alpha = 0.01$ 은 약 **10배 느리다**.
- $\alpha = 0.001$ 은 300스텝이 지나도 바닥에서 꽤
먼 채로 거의 움직이지 못함.
- 1D 그림은 “속도”를 보여준다 — 학습률은
“가는 방향”이 아니라 “**걸음 수**”를 정하는
하이퍼파라미터.

2D 지형: 경로는 왜 일직선이 아닌가



- 등고선이 **늘어진 타원**인 이유 = 앞선 고유값 비율 (≈ 46) 이 바로 그 타원의 축 비율이기 때문.
- 매 스텝 “가장 가파르게 내려가는 방향”을 걷는데, 타원 등고선에서 그 방향은 골짜기 바닥(긴 축)이 아니라 **대각 방향** — 그래서 경로는 짧은 축을 수직으로 오가며 **zigzag**(빨간 점들).
- 등고선이 원형(두 고유값이 같을 때)에 가까우면 진동이 사라져 직선에 가까워짐.

10배 규칙: 실데이터에서는 느림이 흔한 실패모드

장난감(3점) 데이터에선 학습률 비교가 “발산/비발산” 이분법이지만, 실사용 크기에서는 **느린 수렴**이 더 흔하다. $y = 3x + 1 + \text{노이즈}(0, 0.5^2)$ 로 만든 **300개** 데이터, $(0, 0)$ 출발:

학습률	소음 바닥 도달 스텝
$\alpha = 0.001$	3000스텝 내 도달 불가 (최종 $J = 0.154$, 바닥보다 20% 높음)
$\alpha = 0.01$	489
$\alpha = 0.1$	47

소음 바닥 $J_{\text{floor}} = 0.1287$ — 노이즈가 만들어낸, 어떤 직선도 더 낮출 수 없는 이론적 최소 비용.

학습률을 10배 올리면 스텝 수도 약 10배 줄어든다 — 안전한 범위(이 데이터의 경계 $\alpha^* \approx 1.98$) 안에서는 **클수록 비례해서 빨리** 수렴.

학습률의 함정: 너무 작은 값

- α 가 너무 작으면 수렴이 느리다 — 300개 데이터에서 $\alpha = 0.001$ 은 3000스텝을 돌려도 소음 바닥까지 못 닿음.
- **발산하지는 않지만**, “학습 중”이 아니라 “거의 멈춰 있는” 것과 구별하기 어려울 만큼 느리다.

실전 절차

“작을수록 안전”이 아니라, **비용 곡선이 매끄럽게 내려가는 가장 큰 학습률을 찾는 것.**
비용 곡선이 거의 수평으로 뻗어 있다면 발산 문제 이전에 “학습률이 너무 작은 것”부터 의심.

경계는 데이터마다 다르다 (장난감: 0.361, 300개 데이터: 1.98) — 계산 법은 $\alpha^* = 2/\lambda_{\max}$ 로 이미 배웠고, 2.2절에서 스케일에 어떻게 흔들는지 본다.

손으로 한 번: 세 스텝 직접 계산

아주 작은 데이터셋 $X = [1, 2, 3]$, $y = [3, 5, 7]$ (참값 $y = 2x + 1$), $w_0 = w_1 = 0$ 에서 시작, $\alpha = 0.1$.

초기 비용:

$$J(0, 0) = \frac{1}{6} [3^2 + 5^2 + 7^2] = \frac{83}{6} \approx 13.83$$

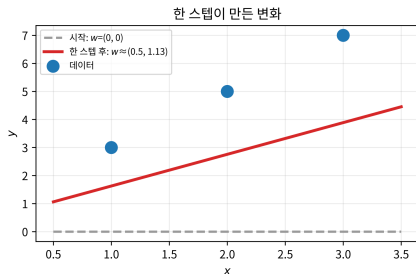
그래디언트:

$$\frac{\partial J}{\partial w_0} = -5, \quad \frac{\partial J}{\partial w_1} \approx -11.33$$

한 스텝 이동:

$$w_0 \leftarrow 0 - 0.1(-5) = 0.5$$

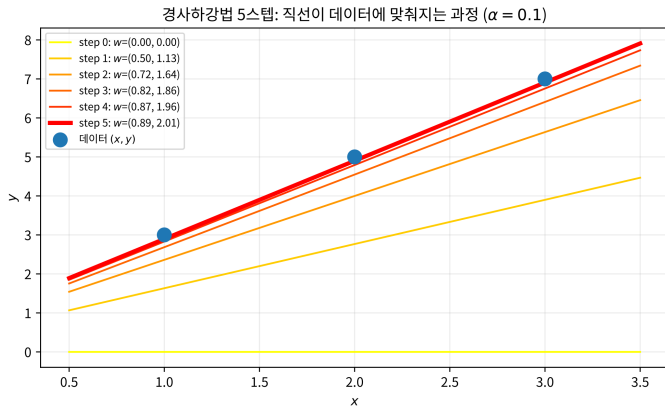
$$w_1 \leftarrow 0 - 0.1(-11.33) \approx 1.13$$



손계산 다섯 스텝의 결과

스텝	w_0	w_1	$J(w)$
0 (초기)	0.0000	0.0000	13.8333
1	0.5000	1.1333	2.7443
2	0.7233	1.6378	0.5448
3	0.8234	1.8621	0.1086
4	0.8687	1.9618	0.0221
5	0.8894	2.0059	0.0049

w_1 이 참값 2에, w_0 이 참값 1에 매 스텝 빠르게 가까워지고 비용이 **단조 감소**한다.



“경사의 반대 방향으로 이동한다”는 말이 실제로 뜻하는 바가 표와 그림이 같이 보여주는 이 과정이다.

자주 하는 실수: 학습률을 너무 크게 잡기

같은 데이터, 같은 초기값에서 $\alpha = 1.5$ 로만:

```
w0, w1 = 0.0, 0.0
alpha_big = 1.5
for i in range(1, 6):
    g0, g1 = grad(w0, w1, X, y)
    w0 -= alpha_big * g0
    w1 -= alpha_big * g1
    print(f"step {i}: w0={w0:.4f} w1={w1:.4f} cost={cost(w0, w1, X, y):.4f}")
```

```
step 1: w0=7.5000 w1=17.0000 cost=741.13
step 2: w0=-47.2500 w1=-107.5000 cost=39708.03
step 3: w0=353.6250 w1=803.7500 cost=2127480.20
step 4: w0=-2580.5625 w1=-5866.3750 cost=113986311.47
step 5: w0=18896.9062 w1=42956.9375 cost=6107168110.25
```

발산의 모습과 실전 대응

- 비용이 줄어드는 커닝 스텝마다 **자릿수가 계속 늘어나며 폭발** — 걸음이 너무 커서 골짜기 바닥을 지나쳐 반대편 비탈로 튕겨 나가고, 다음 스텝엔 그 반대편에서 **더 크게** 튕겨 나오는 반복.

실전 대응

손실이 NaN 이나 매우 큰 값으로 튀면, **가장 먼저 의심할 것이 바로 학습률.**

보통 $\alpha = 0.1, 0.01, 0.001$ 처럼 10배씩 줄여가며 손실 곡선이 매끄럽게 내려가는 지점을 찾는다 — 이 “10배씩” 경험칙이 임의가 아니라 데이터 특이적 경계값 α^* 가 존재하기 때문 (앞 슬라이드 확인).

확인 문제: $\alpha = 0.5$ 는?

$\alpha = 0.1$ 은 매끄럽게 줄었고, $\alpha = 1.5$ 는 발산했다. **실행 전에 먼저 예상하라** — $\alpha = 0.5$ 는 수렴하되 0.1보다 빠를지, 여전히 발산할지, 진동하다가 서서히 수렴할지.

스텝	w_0	w_1	$J(w)$
1	2.5000	5.6667	43.4954
2	-1.9167	-4.3889	136.7638
3	5.9306	13.4352	430.0336
4	-7.9699	-18.1775	1352.1803
5	16.6925	37.8732	4251.7440

정답: $\alpha = 0.5 > \alpha^* \approx 0.361$ 이므로 **발산**. w 가 양수/음수로 **진동**하면서 크기는 계속 커진다.

숫자 0.361 자체가 아니라 **경계값이 존재한다는 것**이 요점: 특징이 $n = 3, 4$ 여도 규칙은 그대로 $0 < \alpha < 2/\lambda_{\max} - \lambda_{\max}$ 는 그냥 그때의 $(n+1) \times (n+1) X^T X / m$ 의 최대 고유값일 뿐, 공식이 바뀌지 않는다.

느리게 발산하는 형태가 더 위험

- $\alpha = 1.5$ 처럼 바로 크게 폭발하지 않고 **느리게 발산**하는 형태.
- “골짜기 바닥을 지나쳐 반대편으로 넘어간다”의 실제 모습 — 진동하면서 성장.

경계값을 넘은 신호

비용이 **“줄어든 뒤 다시 올라가기 시작”**하는 순간. 실전에서 이 패턴을 보면 학습률부터 줄여라.

(힌트 회고) 이 데이터셋에서 발산이 시작되는 경계값은 데이터 스케일에 따라 달라진다 — 특징의 값이 훨씬 크거나 작아지면 같은 α 라도 안전한 범위가 달라진다. 2.2절 특징 스케일링의 중요성 중 하나.

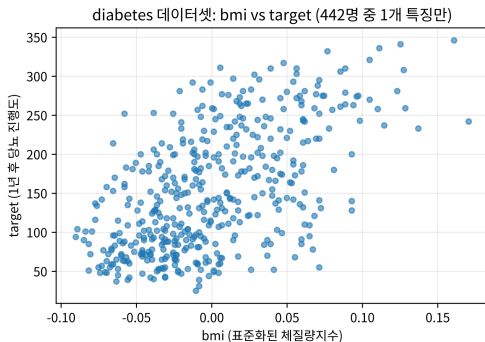
실데이터: diabetes 데이터셋 (1/2)

3개 데이터의 장난감을 넘어, **실전 문제** (특징 10개, 442명)에서 경사하강법의 수렴 모양을 확인.

- sklearn load_diabetes (당뇨 진행도 예측, 10개 신체 특징) 전체 442명 — Efron, Hastie, Johnstone, Tibshirani (2004) 의 데이터.
- 모든 특징을 **표준화**한 뒤 (2.2절 스케일링의 동기) 두 가지를 재본다.

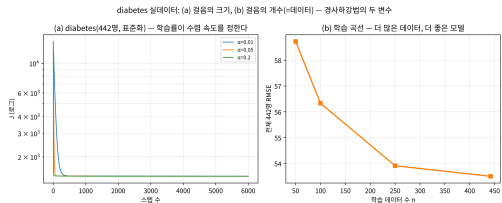
(1) 세 학습률의 비용 곡선 ($\alpha \in \{0.01, 0.05, 0.2\}$):
표준화 후 $(1/m)X^T X$ 의 최대 고유값 $\lambda_{\max} \approx 4.0$
이므로 경계 $\alpha^* = 2/\lambda_{\max} \approx 0.50 - \alpha = 0.2$ 도 경계 안이라 발산하지 않는다.

초기 $J \approx 14537$ 에서 1000스텝 후 J : **1439 / 1435 / 1430**, 6000스텝 후 **1433.9 / 1429.9 / 1429.9** —
클수록 빠르게 바닥에 붙고, “10배 규칙”이 실데이터에서도 대략 성립.



10개 특징 중 하나(bmi)만 골라 본 모습. 출처:
scikit-learn.org/stable/datasets/toy_dataset.html#diabetes-dataset.

실데이터: diabetes 데이터셋 (2/2)



(좌) 세 학습률의 비용 곡선, (우) 데이터 크기 n 에 따른 최종 RMSE의 학습 곡선.

(2) 학습 곡선 — $n = 50, 100, 250, 442$ 로 학습한 모델을 전체 442명 위에서 RMSE로 잰 때:

$$58.7 \rightarrow 56.3 \rightarrow 53.9 \rightarrow 53.5$$

데이터가 작을수록 모델이 **나빠진다** — “배울 것이 적어서”.

수렴 스텝 수는 n 에 대해 단조가 아님 ($2869 \rightarrow 912 \rightarrow 2807 \rightarrow 2405$) — “데이터가 적으면 빠르다”는 일반 법칙이 아니라, 소음 바닥의 크기 자체가 n 에 따라 달라지기 때문.

결과의 핵심 한 줄

핵심

전체 442명으로 학습한 경사하강법의 최종 RMSE **53.5** 는, 같은 데이터의 **정규방정식(닫힌 형태) 해의 RMSE와 일치**.

2.2절에서 “경사하강법과 정규방정식은 같은 목적지”라고 할 때의 말이, 장난감 3점을 넘어 **실데이터 위에서 검증된 셈**.

3개 데이터의 장난감은 할 일을 다했다 — 이제 같은 모델과 최적화 도구로

- “닫힌 형태의 답” (2.2절 정규방정식) 과
- “분류” (2.3절 로지스틱회귀) 로 나간다.

유명한 사례: 가우스와 사라진 소행성 세레스

- 1801년 발견된 소행성 **세레스**가 단 **41일치** 관측(피아치, 1/1~2/11) 뒤 태양 뒤로 사라지자, 24세 **가우스**가 그 부정확한 관측만으로 재출현 위치를 정확히 예측.
- 쓴 방법 = **오차 제곱합 최소화** — 지금 이 절의 바로 그 $J(w)$.

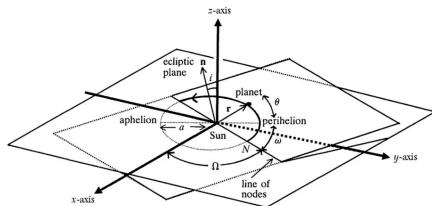


FIGURE 1
Parameters describing the planetary orbit

200년 넘게 쓰이는 이유

“아름다워서”가 아니라, 관측(데이터)이 참값에 잡음이 섞인 구조가 **실존 세계에 끊임없이 반복**되기 때문.

궤도는 타원인데 우리가 배운 건 직선 아닌가? — 최소제곱법이 재는 건 “직선”이 아니라 **예측값과 관측값의 오차 제곱합**. h 가 직선이든 타원 궤도 방정식이든 $J(w) = \sum (h_w - \text{관측})^2$ 구조는 동일 — 가우스는 6개짜리 비선형 h 로 풀었을 뿐, **같은 원리, 더 많은 파라미터**.

궤도를 정하는 6개 숫자 w : a (장반경), e (이심률), i (경사각), Ω (승교점 경도), ω (근일점 인수), θ (위치) — 직선의 (w_0, w_1) 이 6개로 늘어난 것과 본질이 같다.

자주 묻는 질문 (1)

Q. 경사하강법이 지역 최솟값에 갇힐 수 있나요?

- 이 절의 $J(w)$ (평균제곱오차)는 w 에 대한 **볼록함수**라 **지역 최솟값이 곧 전역 최솟값** — 어디서 출발하든 결국 같은 최적점에 수렴.
- 지역 최솟값이 실제 골칫거리가 되는 건 **Week 9 신경망처럼 비볼록 손실**을 쓸 때.

Q. 매 스텝 데이터 전체를 다 봐야 하나요?

- 위 코드 = 매 스텝 m 개 전부로 그래디언트를 계산하는 **배치 경사하강법**(batch GD).
- 데이터가 수백만 개면 스텝 하나조차 느려지는데, 실전은 작은 묶음(mini-batch)만 보는 **확률적 경사하강법**(SGD)을 훨씬 더 많이 쓴다 — Week 9에서 변형들을 자세히.

자주 묻는 질문 (2)

Q. 학습률은 실전에서 어떻게 정하나요?

- J 가 이차함수이면 (선형회귀) 이 절의 공식으로 정량적:

$$0 < \alpha < 2/\lambda_{\max}$$

($\lambda_{\max} = (1/m)X^T X$ 의 최대 고유값).

- 신경망처럼 이차함수가 아니면 공식이 없으므로, 보통 $\alpha = 0.1 \sim 0.01$ 에서 시작해 비용 곡선을 보고 조절: 매끄럽게 내려가면 더 큰 값으로, 흔들리거나 발산하면 10배 줄인다.
- 파라미터마다 학습률을 자동 조절하는 **적응형 학습률 알고리즘** (SGD 변형, Adam 등)이 실전의 표준 — Week 9.

Q. 0에서 시작하는 것은 필수인가?

- 아니요 — 볼록함수는 **어디서 시작하든** 같은 최솟값에 도달 (첫 Q 참고). 0은 관례일 뿐.
- 단, 초기값이 멀수록 스텝 수 $\uparrow$ — 특징 표준화 (2.2절)가 실용적인 이유.

2.2 정규방정식과 “같은 모델, 다른 출력”이라는 다리

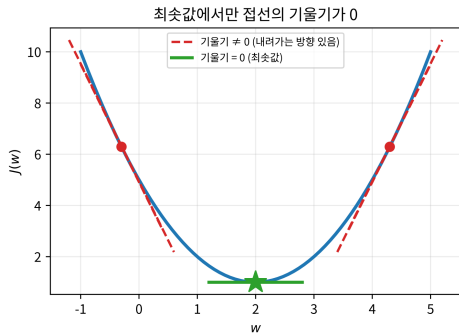
정규방정식: 미분으로 한 번에 풀기

$J(w)$ 는 w 에 대한 이차함수이므로, 경사하강법 없이 **미분을 0으로 놓아 닫힌 형태(closed-form)로 최적해를 구할** 수도 있다:

$$\nabla_w J = \frac{1}{m} X^T (Xw - y) = 0 \Rightarrow X^T X w = X^T y$$

$$\Rightarrow w^* = (X^T X)^{-1} X^T y$$

- X : 각 행이 $x^{(i)}$ (bias 포함)인 $m \times (n + 1)$ 행렬
- $\nabla_w J = 0$ 을 직접 전개해 위 두 줄을 얻는 과정이 이번 장 연습문제(2번)의 핵심.



볼록함수는 접선 기울기가 0인 곳이 곧 최솟값.

경사하강법 대 정규방정식

	정규방정식	경사하강법
반복	불필요 — 정확	학습률 튜닝, 스텝 수 필요
계산 비용	$(X^T X)^{-1}$ — 특징 개수 n 의 세제곱 에 비례	n 이 커져도 매 반복 비용은 선형 으로만 증가
쓰이는 곳	특징이 적 을 때	특징이 많 을 때 (수천~수만)

특징이 적을 땐 정규방정식, 많을 땐 경사하강법을 쓰는 이유다.

손으로 한 번: 정규방정식으로 직접 풀어보기 (1/5)

2.1절에서 경사하강법으로 다섯 스텝 만에 $w_0 \approx 0.89$, $w_1 \approx 2.01$ 에 근접했던 그 데이터셋 $X = [1, 2, 3]$, $y = [3, 5, 7]$ 을 정규방정식으로 **한 번에** 풀어보자.

bias 열을 추가한 행렬:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$$

손으로 한 번: 정규방정식으로 직접 풀어보기 (2/5)

2.1절에서 경사하강법으로 다섯 스텝 만에 $w_0 \approx 0.89$, $w_1 \approx 2.01$ 에 근접했던 그 데이터셋 $X = [1, 2, 3]$, $y = [3, 5, 7]$ 을 정규방정식으로 **한 번에** 풀어보자.

bias 열을 추가한 행렬:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$$

$X^T X$ 계산:

$$X^T X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}$$

손으로 한 번: 정규방정식으로 직접 풀어보기 (3/5)

2×2 행렬 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 의 역행렬 공식 $\frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ 적용: $ad - bc = 3 \times 14 - 6 \times 6 = 6$ 이므로:

$$(X^T X)^{-1} = \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} = \begin{bmatrix} 2.333 & -1 \\ -1 & 0.5 \end{bmatrix}$$

손으로 한 번: 정규방정식으로 직접 풀어보기 (4/5)

2×2 행렬 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 의 역행렬 공식 $\frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ 적용: $ad - bc = 3 \times 14 - 6 \times 6 = 6$ 이므로:

$$(X^T X)^{-1} = \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} = \begin{bmatrix} 2.333 & -1 \\ -1 & 0.5 \end{bmatrix}$$

$$X^T y = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix} = \begin{bmatrix} 15 \\ 34 \end{bmatrix}$$

손으로 한 번: 정규방정식으로 직접 풀어보기 (5/5)

2×2 행렬 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 의 역행렬 공식 $\frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ 적용: $ad - bc = 3 \times 14 - 6 \times 6 = 6$ 이므로:

$$(X^T X)^{-1} = \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} = \begin{bmatrix} 2.333 & -1 \\ -1 & 0.5 \end{bmatrix}$$

$$X^T y = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix} = \begin{bmatrix} 15 \\ 34 \end{bmatrix}, \quad w^* = \begin{bmatrix} 2.333 & -1 \\ -1 & 0.5 \end{bmatrix} \begin{bmatrix} 15 \\ 34 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$w^* = [1, 2]$ — **정확히** 데이터를 만들 때 쓴 참값 $y = 2x + 1$ 과 일치. 경사하강법은 “근접”했을 뿐, 정규방정식은 **단 한 번의 행렬 계산으로 정확히** 도달.

실습: numpy로 정규방정식 → GD와 비교

```
import numpy as np

X_raw = np.array([1.0, 2.0, 3.0])
y = np.array([3.0, 5.0, 7.0])
m = len(X_raw)
X = np.column_stack([np.ones(m), X_raw]) # bias 열추가

w_star = np.linalg.inv(X.T @ X) @ X.T @ y
print("정규방정식 해:", w_star)           # [1. 2.]

# 경사하강법으로 충분히 오래 스텝 (2000) 돌리면 같은 값에
w0, w1, alpha = 0.0, 0.0, 0.1
for _ in range(2000):
    pred = w0 + w1 * X_raw
    err = pred - y
    w0 -= alpha * np.sum(err) / m
    w1 -= alpha * np.sum(err * X_raw) / m
print("GD 스텝(2000):", w0, w1)          # 1.0, 2.0
```

두 방법은 왜 같은 지점에 도달할까

핵심

두 방법이 결국 같은 지점에 도달하는 것은 **우연이 아니다** — $J(w)$ 가 **볼록함수**라서 유일한 전역 최솟값을 가지므로, “미분해서 한 번에 푼다”와 “조금씩 내려간다”는 서로 다른 두 길이 **같은 목적지**로 이어지는 것뿐이다.

행렬로 다시 쓰기: 닫힌 형태 해

$J(\mathbf{w}) = \frac{1}{2m} \sum_i (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)})^2$ 를 벡터 $y = (y^{(1)}, \dots, y^{(m)})^T$ 로 쓰면 행렬곱으로 단축:

$$J(\mathbf{w}) = \frac{1}{2m} \|X\mathbf{w} - y\|^2$$

미분은 두 줄 (연쇄법칙: 힌트는 연습문제 2):

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m} X^T (X\mathbf{w} - y)$$

이를 0 으로 놓고 $X^T X \mathbf{w} = X^T y$ 로 정리하면 $\mathbf{w}^* = (X^T X)^{-1} X^T y$ — **정규방정식(normal equations)**. $(X^T X)$ 가 역행렬 가능한 조건은 다음 “자주 하는 실수”가 다룬다.

사영: 왜 이름이 “normal(직각)”인가

이 유도에서 중요한 건 수식이 아니라 **최해의 기하학적 뜻**:

- Xw 는 어떤 w 를 넣어도 X 의 열들이 펼치는 부분공간 (column space) 안에만 살아 있다.
- “최소제곱으로 y 에 가장 가까운 Xw ” = 이 부분공간 안에서 y 에 가장 가까운 점 = 정확히 y 의 사영(projection).
- “가장 가까운 점”은 직각인 방향에서 찾는다: 최적점에서 오차 벡터 $Xw^* - y$ 가 X 의 모든 열에 수직:

$$X^T(Xw^* - y) = 0$$

이 한 줄이 정규방정식 그 자체. X^T 로 곱한다는 동작이 “모든 특징에 대해 수직”을 보장 — 이름 “normal”(직각)은 이 수직성에서 온 것.

조건수: “늘어난 골짜기”를 재는 숫자

J 가 이차함수이므로 $\nabla J = \frac{1}{m} X^T X (\mathbf{w} - \mathbf{w}^*)$:

$$\mathbf{w}^{(t+1)} - \mathbf{w}^* = \left(I - \frac{\alpha}{m} X^T X \right) (\mathbf{w}^{(t)} - \mathbf{w}^*)$$

- $I - \frac{\alpha}{m} X^T X =$ 오차를 줄여주는 곱셈 행렬. $X^T X$ 의 고유값 $\lambda_1 \leq \lambda_2$ (2×2) 이면, 매 스텝 두 방향의 오차가 $(1 - \frac{\alpha \lambda_i}{m})$ 배로 줄어듦.
- 수렴 속도는 **더 큰 고유값 방향** — 등고선이 더 늘어지는 방향을 재고, 이 “늘어짐”의 크기가 **조건수** $\kappa = \lambda_{\max} / \lambda_{\min}$.

이 데이터셋 ($x = [1, 2, 3]$) 에서 $\kappa \approx 46.1$.

$\kappa \approx 46$ vs $\kappa \approx 2$: 같은 골짜기가 아님

$$x = [1, 2, 3]$$

“균형 잡힌” 데이터 (100 개, $x \sim \mathcal{N}(0, 1)$)

조건수 $\kappa \approx 46.1$

≈ 2

등고선 긴 축: 짧은 축 $\approx 46:1$ 타원 — **zigzag**하며 천천히 하강

거의 원형 — 직선에 가까운 경로

스케일 10배가 조건수를 ~ 71 배로 만들면

같은 데이터에 특징 스케일 10배 ($x = [10, 20, 30]$, 참값 $y = 0.2x + 1$ — 직선은 **같음**) 로 바꾸면 $\kappa : 46.1 \rightarrow 3278.7$ — 약 **71배** 커짐. (bias 열은 스케일링되지 않아 정확히 100배는 아니고, 두 고유값이 스케일에 따라 다르게 늘어나 비율이 바뀜.)

κ 가 걸음 수를 정한다: 매 스텝 오차 감소율

매 스텝 오차 감소율은 $(\kappa - 1)/(\kappa + 1)$:

	$\kappa = 3278$	$\kappa = 2$
매 스텝 남는 오차 비율	≈ 0.9994 — 100스텝에도 오차의 약 60% 남음	$= 0.333$ — 100스텝이면 오차가 1

정량적 용언

2.1절 “학습률의 함정”이 스케일에 민감하다는 것의 **정량적 용언**: 특징 스케일이 30배로 다르면, 같은 α 로 수렴하는 데 걸리는 스텝 수가 **수만 배**로 달라질 수 있다.
특징 스케일링이 “편의”가 아니라 **경사하강법의 걸음 수를 바꾸는 핵심 전처리**인 이유. (Week 4에서 다시 만남)

특징 스케일링: 골짜기를 “등근” 형태로

실습 노트북 § 3: 참값 $y = 1.5x + 20$, 데이터 $x \in [50, 130]$ 5개 — **같은 문제**, 스케일링만 차이.

	raw 스케일 ($x = 50 \sim 130$)	표준화 $((x - \mu)/\sigma)$
조건수	≈ 99033	≈ 1.0
안정 학습률	$\alpha < 2/\lambda_{\max} \approx 0.00022$ 근처의 $\alpha = 0.0002$	$\alpha < 2$ 근처의 $\alpha = 0.5$
$J < 10^{-6}$ 에	464,595 스텝	18 스텝

스케일링 한 개가 학습 스텝을 약 **25,800배** 줄였다.

스케일링은 답을 바꾸는 게 아니다

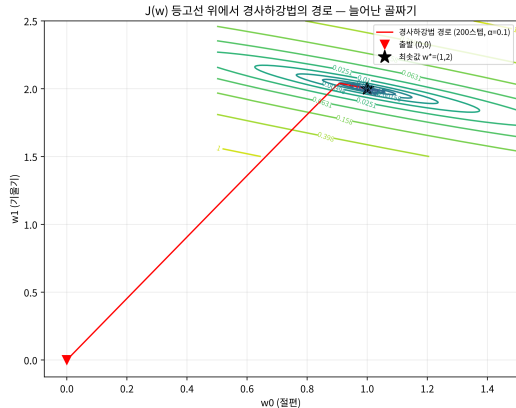
- 정규방정식은 두 경우 모두 **한 번의 행렬 계산으로 정확히** ($b = 20, \text{slope} = 1.5$) 를 낸다.
- 표준화 케이스는 $b = 155, \text{slope}_s = 42.43$ 이 나와도, 스케일 변환으로 역변환하면 $\text{slope} = 42.43/\sigma = 1.5$ 가 된다.

이 절에서 꼭 잡아야 할 핵심

스케일링은 답을 바꾸는 게 아니라 경사하강법의 걸음 수를 바꾸는 것.

정규방정식은 스케일링 여부와 무관하게 같은 답(단, 스케일이 바뀌면 계수 스케일도 같이 바뀜)를 주므로, “답이 바뀌어서” 스케일링을 하는 게 아니다.

두 경로, 같은 최솟값



같은 데이터, 같은 출발점에서 정규방정식(직선)과 경사하강법(빨간, 200스텝)이 같은 최솟값 (1, 2) 에 도달.

- 등고선이 w_0 축으로 늘어난 타원 — 이 데이터셋의 $\kappa \approx 46$ 에 대응.
- 경사하강법은 골짜기 바닥을 일직선으로 타지 못하고 **짧은 축 방향으로 진동**하며 천천히 하강.
- 조건수 46인 골짜기에서 $(\kappa - 1)/(\kappa + 1) \approx 0.958$ — 매 스텝 오차의 95.8%가 남으므로, 200스텝이 “느리게” 보이는 것은 이 수치에서 나온다.

Gauss-Markov 정리: 최소제곱이 “최선”인 이유

- 최소제곱 해(정규방정식의 해)는 “편하게 풀 수 있는” 해가 아니라 **특정 의미에서 최선** — 가우스(1821)가 먼저 증명, 마코프(1905)가 일반적 형태로 다시 증명해 이름이 합쳐짐.

Gauss-Markov 정리

선형모형 $y = Xw + \varepsilon$ 에서 오차 ε 의 기댓값이 0이고 분산이 균일(σ^2 으로 동일)하며 상호 독립이면, **최소제곱 추정치(OLS, 정규방정식의 해)는 모든 선형 무편차 추정치 중 분산이 가장 작은 것** (BLUE — Best Linear Unbiased Estimator).

“최선”의 경계와 실전 의미

- “최선”의 기준은 **선형 무편차** 추정치라는 범주 안에서만 성립. 비선형 추정치나 편이 있는 추정치 (Week 6의 정규화는 의도적으로 편이를 만들어 분산을 줄이는) 는 대상이 아님.
- 실전 의미: 최소제곱 해가 “정답”이 아니라도, **“어느 선형 무편차 방법으로도 분산을 더 줄일 수는 없다”**는 보장이 있다 — “왜 이 방법으로 풀까?”에 “수학적으로 최선”이라 답할 근거.

Gauss-Markov: “선형 무편차 안에서 최선” — 그 경계를 정확히 말해야 한다.

역사: 가우스가 정규방정식을 발견한 자리

세레스 궤도 문제(2.1절) 에피소드 뒤에는 **정규방정식 그 자체**의 발견이 있다.

- 가우스는 “관측 y 가 Xw 의 선형 결합에 오차 ε 이 섞인 구조”를 인식하고, $X^T X$ 가 대칭 행렬이고 이 대칭 행렬을 풀어 w^* 를 구하는 과정을 정식화 — 바로 $w^* = (X^T X)^{-1} X^T y$.
- **최소제곱 “손실함수”**와 그를 최적화하는 **“달힌 형태 해”**가 **같은 시기에, 같은 문제에서 발견**되었다는 것이 중요.

손실함수 vs 최적화 알고리즘 — 분리될 수 있는 두 작업

르장드르(1805, 발표)는 최소제곱 손실함수를, 가우스(1809, 『천체 궤도 이론』)는 최소제곱 해의 계산법을 먼저 정식화.

로지스틱회귀(2.3절)와 신경망(Week 9)에서는 이 둘이 **분리된다** — 교차 엔트로피는 이차함수가 아니라서 달힌 형태 해가 없고, 경사하강법이 독립적으로 발견된 알고리즘이 손실을 최소화.

자주 하는 실수: 특징이 겹쳐 행렬이 특이해지면

정규방정식은 $(X^T X)^{-1}$ 이 **존재할 때만** 계산 가능. 두 번째 특징이 첫 번째의 배수(완전 선형종속, 예: “미터 단위 넓이” + “제곱피트 단위 넓이”) 면:

```
X2_raw = np.column_stack([X_raw, 2 * X_raw])    # 두번째 = 첫번째의배 2
X2 = np.column_stack([np.ones(m), X2_raw])
print("det(X^T X) =", np.linalg.det(X2.T @ X2))  # 0.0
np.linalg.inv(X2.T @ X2)    # LinAlgError: Singular matrix
```

행렬식이 정확히 0 — **다중공선성**(multicollinearity) 으로 정보가 w 의 두 성분 중 어느 쪽에 어떻게 배분돼야 할지 수학적으로 결정할 방법이 없어져, **무한히 많은** (w_1, w_2) 조합이 똑같은 예측을 내놓을 수 있게 된다.

해결: 무어-펜로즈 준역행렬 (pinv)

inv 대신 유한한 값을 돌려주는 np.linalg.pinv (**무어-펜로즈 준역행렬**, Moore-Penrose pseudoinverse):

```
w_pinv = np.linalg.pinv(X2.T @ X2) @ X2.T @ y
print(w_pinv)           # [1.  0.4 0.8]
print(X2 @ w_pinv)      # [3.  5.  7.] —정확히들어맞음

w_alt = np.array([1.0, 1.0, 0.5]) # 똑같은예측을내는다른 ** 해
print(X2 @ w_alt)           # [3.  5.  7.]
print("||w_pinv||^2 =", w_pinv @ w_pinv, " ||w_alt||^2 =", w_alt @ w_alt)
# 1.8 vs 2.25
```

두 해 모두 세 데이터를 정확히 맞는데, pinv가 고른 해의 $\|w\|^2$ 가 더 작다 — pinv는 무한히 많은 해 중 $\|w\|^2$ **가 가장 작은 해**(최소범위 해, minimum-norm solution)를 골라준다.

특이 행렬의 실전 해석과 근종속

- **실전 해석 시 주의:** 완전히 중복된 특징이 섞인 모델에서 w_1, w_2 각각의 값은 “진짜” 계수가 아니라 **배분의 산물** — 그런 특징끼리는 **하나를 골라 버리는 것**이 해석을 위한 정석적 해결.
- 실전에서 LinAlgError 나, det 가 0에 아주 가까워 **조건수가 거대**하면, “특징 중 서로 거의 같은 정보를 담은 게 있는지”부터 점검이 **첫 단추**.
- “완전” 종속보다 **“거의” 종속** (상관계수 0.99) 이 더 흔하고 더 슬프다 — 역행렬은 존재하지만 조건수가 10^{15} 을 넘으면 float64 유효숫자가 바닥나 w 가 미세한 흔들림에 **거대하게** 요동.

경사하강법은 에러 없이 돌아가지만 w 가 매우 불안정 — Week 6의 정규화(regularization)가 이런 상황을 다스리는 정식 도구 (ℓ_2 정규화가 조건수를 기계적으로 다듬는 이유도 6.1절 확인).

회귀에서 분류로: 같은 도구를 다른 문제에

- 로지스틱회귀라는 이름 때문에 헷갈리기 쉽지만, 이건 회귀가 아니라 **분류** 알고리즘.
- 선형회귀의 출력 $w^T x$ 는 $-\infty$ 부터 $+\infty$ 까지 아무 값이나 되는데, “스팸일 확률”은 반드시 0과 1 사이여야 한다.

핵심 아이디어

2.1절의 **모델**과 **경사하강법**을 그대로 재사용하고, “**출력을 확률로 바꾸는 변환**”과 “**그 확률에 맞는 새 손실함수**”만 추가하면 된다 — 이번 장 후반부의 핵심.

“다리”를 한 눈으로: 비교 표

단계	선형회귀 (2.1)	로지스틱회귀 (2.3)
모델	$h_w(x) = w^T x$	$h_w(x) = \sigma(w^T x)$
출력의 의미	연속 숫자 (예: 방 가격)	확률 (0~1)
손실함수	평균제곱오차 $\frac{1}{2m} \ Xw - y\ ^2$	교차 엔트로피
손실이 w 에 대해	이차함수 (볼록) $\rightarrow$ 정규방정식 존재	비이차(볼록) $\rightarrow$ 닫힌 형태 해 없음
최적화	정규방정식 또는 경사하강법	경사하강법만
그라디언트	$\frac{1}{m} X^T (Xw - y)$	$\frac{1}{m} X^T (\sigma(Xw) - y)$ — 형태 동일

“그래디언트 형태 동일”이 뜻하는 것

- 마지막 줄(“형태 동일”)은 2.3절에서 **증명**할 내용. 미리 눈여겨봐두자: **시그모이드의 미분**
 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ **가 교차 엔트로피 손실의 미분과 만나서 통째로 약분**되어, 최종
그래디언트가 선형회귀와 똑같은 $(h - y) \cdot x$ 형태가 된다.

“같은 도구를 다른 문제에 쓴다”의 구체적 뜻

경사하강법의 코드 줄 자체를 그대로 복사해서, h 를 계산하는 부분에 시그모이드 한 줄만 추가하는 것.

이 패턴 — **모델 구조는 그대로 두고, 출력 해석과 손실함수만 문제에 맞게 바꾼다** — 는 학기 내내 반복.

2.3절 = 이 패턴의 첫 완전한 예(회귀→분류). Week 9 신경망은 “출력 변환”을 여러 층으로 늘린 것, Week 15 생성모형은 같은 파이프라인을 “생성”이라는 새 문제에 맞춘 것.

확인 문제: 이 절의 세 가지

- ❶ **왜 둘은 같은 점에 도달할까?** 경사하강법 2000스텝과 정규방정식 1회가 같은 w^* 를 준다. $J(w)$ 의 어떤 성질이 “어디서 출발하든 같은 목적지”를 보장하는가? (힌트: 2.1절 FAQ의 볼록함수.)
- ❷ **왜 조건수가 걸음 수를 정할까?** $x = [1, 2, 3]$ 의 $\kappa \approx 46$ 과 $x = [10, 20, 30]$ 의 ≈ 3279 를 대입하면 매 스텝 오차 감소율 $(\kappa - 1)/(\kappa + 1)$ 은 각각 ≈ 0.958 과 ≈ 0.9994 . $\kappa = 3279$ 일 때 오차가 **절반**으로 줄어드는 데 몇 스텝인가? (힌트: $0.9994^t = 0.5$ — 답 **약 1140스텝**. $\kappa = 46$ 이면 약 16스텝, 약 71배 차이.)
- ❸ **사영과 수직성.** $X^T(Xw^* - y) = 0$ 이 “오차 벡터가 X 의 모든 열에 수직”을 뜻하는 이유를 한 문장으로. “normal(직각)” 이름의 출처를 떠올려라.

자주 묻는 질문 (1)

Q. 정규방정식이 있는데 왜 경사하강법을 배우나요?

- 정규방정식은 선형회귀처럼 $J(w)$ 가 이차함수라서 미분=0을 대수적으로 풀 수 있는 **특수한 경우**에만 존재.
- 2.3절 로지스틱회귀부터는 손실(교차 엔트로피)이 w 에 대해 이차함수가 아니라서 **애초에 닫힌 형태 해가 없.**
- Week 9 신경망은 손실이 **비볼록**까지.
- → 경사하강법(과 변형들)이 이 학기 전체를 통틀어 실제로 쓰이는 **범용 도구**. 정규방정식은 “정답과 비교해 경사하강법이 맞게 수렴하는지 검사”하는 용도로 특히 유용.

자주 묻는 질문 (2): inv vs solve

Q. 실전에서 `np.linalg.inv` 를 그냥 쓰면 되나요?

권장

수치적으로는 `np.linalg.solve(X.T @ X, X.T @ y)` 가 역행렬을 명시적으로 구하는 것보다 **더 안정적이고 빠르다** — 역행렬을 구한 뒤 곱하는 것은 **필요 이상의 계산과 오차**를 더한다.
이 절에서는 유도 과정을 눈으로 보여주기 위해 `inv` 를 그대로 썼지만, 실무 코드는 `solve` 습관을.

2.3 로지스틱회귀: 시그모이드에서 PR-AUC까지

시그모이드: 실수를 확률로

로지스틱함수(시그모이드)는 정확히 “ $-\infty \sim +\infty$ 를 $0 \sim 1$ 로 바꾸는” 변환:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $z \rightarrow +\infty \Rightarrow \sigma \rightarrow 1$, $z \rightarrow -\infty \Rightarrow \sigma \rightarrow 0$, $\sigma(0) = 0.5$.
- 어떤 실수든 이 함수를 통과하면 $0 \sim 1$ — “확률”로 해석 가능.

-

$$h_w(x) = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

— “ x 가 **양성 클래스(1)**일 확률”로 해석.

결정 경계는 여전히 직선

- 예측은 $h_w(x) \geq 0.5$ 면 클래스 1, 아니면 클래스 0.
- $h_w(x) = 0.5$ 는 정확히 $w^T x = 0$ 인 지점 → **결정 경계(decision boundary)**는 여전히 **직선**(또는 초평면).

역사의 한 귀퉁이: 뉴런의 수식 (1943)

매컬록-피츠 뉴런 (McCulloch & Pitts, 1943) — “흥분 입력이 임계값을 넘으면 신호를 보내고, 아니면 보내지 않는 0/1 기계”. 뉴런의 실제 출력은 연속적이고 확률적이지만, 이 모델은 수학적으로 다루기 쉬운 첫 뉴런 모델이었다. 그 모델에서 임계값 초과 강도를 “연속적 확률”로 부드럽게 바꿔준 것이 바로 **시그모이드** — 뉴런의 활동전위(activity potential)가 시간에 따라 올라가는 S자 모양을 본떠. (Rosenblatt(1958) 퍼셉트론 이후 활성화함수로 정설 → Minsky(1969)의 『Perceptrons』 비판까지 — Week 15에서 다시.)

log-odds 직관: 시그모이드는 “대수적 기울기”

시그모이드를 “반전”하면 이 변환이 왜 자연스러운지 설명해진다. $p = \sigma(z)$ 를 z 에 대해 정리:

$$z = \log \frac{p}{1-p} = \log \text{odds}(p)$$

- $\text{odds}(p) = p/(1-p) = \text{승산(odds)}$ — “양성일 가능성 : 음성일 가능성”의 비율.
- 시그모이드의 역함수가 **log-odds**(로지트, logit)이므로, 로지스틱회귀는 실제로 **log-odds를 선형 모델로 설명**:

$$\log \frac{h_w(\mathbf{x})}{1-h_w(\mathbf{x})} = \mathbf{w}^T \mathbf{x}$$

숫자: $\mathbf{w}^T \mathbf{x} = 1.0986$ 이면 $h_w(\mathbf{x}) = 0.75$ — 승산 $0.75/0.25 = 3$ 이므로 “양성일 가능성은 음성의 3배”.

계수의 해석: 강도와 방향

- 특징 x_j 의 계수 w_j = “다른 게 다 같을 때 x_j 를 1 올리면 **log-odds가 w_j 만큼** 바뀐다” — 승산이 e^{w_j} 배만큼 변한다.
- 이 해석 덕분에 로지스틱회귀 계수는 “이 특징이 양성을 얼마나 밀어올리는가”의 **강도와 방향**을 읽을 수 있는, **해석 가능한 모델**로 쓰인다.

의학 표준의 이유

의학적 위험인자 분석에서 로지스틱회귀가 **수십 년간 표준**인 이유.

(Week 7의 결정트리와 비교: 트리 경계는 “ x_1 이 5.3 이상”처럼 계수 해석을 주지 않는다.)

손실함수: 평균제곱오차로는 안 되는 이유

- 시그모이드에 평균제곱오차를 그대로 적용하면 $J(w)$ 가 w 에 대해 **볼록하지 않아(non-convex)**, 경사하강법이 **지역 최솟값에 갇힐 위험**이 있다.
- 대신 **교차 엔트로피**(cross-entropy) 손실을 쓴다 — 이 이름은 **1948년 클로드 새넨(Claude Shannon)의 정보이론**(information theory)에서 왔다.

새넨의 정보량: 확률 p 로 일어나는 사건이 실제로 일어났다는 소식의 “놀라움”을 $-\log_2 p$ (단위: 비트)로 정의 —

- 자주 일어나는 사건 ($p \approx 1$): “해가 동쪽에서 떴다” — 정보량 ≈ 0 .
- 거의 안 일어나는 사건 ($p \approx 0$): “복권에 당첨됐다” — 정보량 **크다**.

엔트로피, 교차 엔트로피, KL 발산

- 정보량의 **기댓값** = **엔트로피**(entropy):

$$H(p) = - \sum_k p_k \log_2 p_k$$

분포 p 를 따르는 사건들을 평균적으로 관찰할 때 얻는 정보량이자, 이 분포를 **최적 부호화(encoding)** 하는 데 필요한 평균 비트 수 (Week 7.2 결정트리에서 “가장 잘 나누는 질문” 기준으로 다시 만남).

- 실제 분포는 p 인데, (틀린) 다른 분포 q 를 믿고 부호화하면 필요한 **평균 비트 수** = **교차 엔트로피**:

$$H(p, q) = - \sum_k p_k \log_2 q_k$$

- $q = p$ 일 때 $H(p, q) = H(p)$ 로 **최솟값**. q 가 p 에서 멀어질수록 커지고, 그 초과분 $H(p, q) - H(p) =$ **KL 발산**($D_{KL}(p||q)$) (Week 15 VAE의 ELBO에서 다시 만남).

교차 엔트로피를 손실함수로: 직관

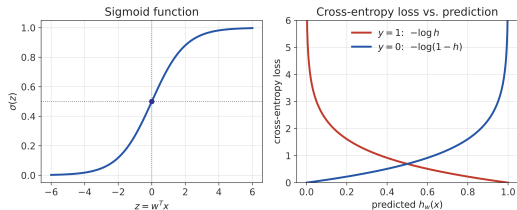
- 로지스틱회귀에서 정답 $y^{(i)}$ = “정답 분포” (클래스 1 확률 100% 또는 0%), $h_w(x^{(i)})$ = 모델이 예측한 분포.
- **교차 엔트로피를 최소화하는 것 = 모델의 예측 분포를 정답 분포에 최대한 가깝게 만드는 것**
— 손실함수로 쓰는 이유:

$$J(w) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_w(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_w(x^{(i)})) \right]$$

- ML에서는 보통 $\log_2$ 대신 **자연로그** $\ln$ — 상수배($1/\ln 2$)만 차이.

직관: 정답 $y = 1$ 인데 $h_w(x) \rightarrow 0$ 으로 확신하면 $-\log h_w(x) \rightarrow \infty$ — **확신을 갖고 틀리면 그만큼 크게 벌한다.**

시그모이드와 교차 엔트로피 손실



시그모이드 함수(좌)와 예측 확률에 따른 교차 엔트로피 손실(우) — 정답과 반대로 확신할수록 손실이 무한히 커진다.

- 정답 $y = 1$ 인데 $h_w(x) \rightarrow 0$ 으로 확신하면 $-\log h_w(x) \rightarrow \infty$ — 손실이 **무한히** 커짐.
- **확신을 갖고 틀리면 그만큼 크게 벌.**

신기하게도 이 손실을 미분하면 2.1절 선형회귀와 **똑같은 형태**의 그레디언트:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m \left(h_w(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

→ 경사하강법 구현은 2.1절과 거의 동일.

로지스틱 경사하강법: 시그모이드 한 줄만 추가

```
import math

def sigmoid(z):
    return 1 / (1 + math.exp(-z))

def logistic_gradient_descent(X, y, alpha, epochs):
    m, n = len(X), len(X[0])
    w = [0.0] * (n + 1)
    for _ in range(epochs):
        grad = [0.0] * (n + 1)
        for i in range(m):
            pred = sigmoid(w[0] + sum(w[j+1] * X[i][j] for j in range(n)))
            error = pred - y[i]
            grad[0] += error
            for j in range(n):
                grad[j+1] += error * X[i][j]
        for j in range(n + 1):
            w[j] -= alpha * grad[j] / m
    return w
```

2.1절 코드와 **차이 = sigmoid() 한 줄**. “오차 × 특징” 패턴이 시그모이드를 통과해도 살아남는 것을 2.3절이 확인.

확인 문제 1: 시그모이드의 미분

- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ (연습문제 5의 힌트).
- $0 < \sigma(z) < 1$ 에서 항상 **양수**, $\sigma(z) = 0.5$ (즉 $z = 0$) 에서 **최대값 0.25**.
- 실습 노트북 § 3: 수치미분 vs 해석미분 비교, 최대 오차 $\approx 2 \times 10^{-4}$ 로 확인.

직관

시그모이드는 “가장 많이 틀린 예측(확률 0.5)에서 가장 민감하게 반응하고, 이미 확신을 가진 예측(0 또는 1에 가까움)에서는 거의 움직이지 않는다” — 그래서 경사하강법이 학습 초반에는 빠르게, 후반에는 천천히 수렴한다.

손으로 한 번: 3샘플 로지스틱회귀, 단계 0

한 특징 $x \in \{1, 2, 3\}$, 정답 $y = [1, 0, 1]$, 초기값 $w_0 = w_1 = 0$, 학습률 $\alpha = 0.5$.

단계 0: 모든 예측 $h = \sigma(0) = 0.5$.

$$J = -\frac{1}{3} [2 \log 0.5 + \log 0.5] = -\log 0.5 = 0.6931$$

단계 1: 그래디언트 —

$$\frac{\partial J}{\partial w_0} = \frac{1}{3} \sum_i (h_i - y_i) = \frac{1}{3} [(0.5 - 1) + (0.5 - 0) + (0.5 - 1)] = -0.1667$$

$$\frac{\partial J}{\partial w_1} = \frac{1}{3} \sum_i (h_i - y_i) x_i = \frac{1}{3} [(0.5 - 1)(1) + (0.5 - 0)(2) + (0.5 - 1)(3)] = -0.3333$$

3샘플 로지스틱회귀: 단계 1 업데이트와 해석

업데이트: $w_0 = 0 - 0.5(-0.1667) = 0.0833$, $w_1 = 0 - 0.5(-0.3333) = 0.1667$.

이제 예측이 더 이상 0.5가 아니다:

$h(1) = \sigma(0.0833+0.1667) = 0.5619$, $h(2) = \sigma(0.0833+0.3333) = 0.5381$, $h(3) = \sigma(0.0833+$

손실은 **0.6475** 로 떨어짐.

해석

두 샘플 ($x = 1, 3$) 은 양성이므로 모두 $h > 0.5$ 로 “정답 쪽”. 유일한 음성 ($x = 2$) 은 가장 작은 기울기로 0.5를 겨우 넘겨 예측.

“반쯤 맞고 반쯤 틀린 예측”을 받으면 그래디언트도 그만큼 작아진다 — 이것이 직관.

실습 노트북 § 2 (numpy, 300 epoch): $w = [-1.494, 1.345, 0.608]$, 최종 손실 0.3626 — sklearn `LogisticRegression(C=1e6)` 와 거의 같은 $[-1.573, 1.492, 0.662]$ 확인.

“정확도”만으로는 안 되는 이유: 정확도의 함정

- 암 검진 모델: 전체 환자의 **99%가 정상**라면, “무조건 정상”이라고만 찍는 모델도 **정확도 99%** — 하지만 암 환자를 **단 한 명도** 잡아내지 못하는 **쓸모없는 모델**.

	환자 1000명 중 10명(1%) 실제 암	모델: 무조건 “정상”
정확도	$990/1000 = 99\%$ (매우 높아 보임)	
Recall	$0/10 = 0\%$	실제 암을 단 한 명도 못 잡아

“정확도의 함정(accuracy paradox)”: 클래스 불균형 상황에서 정확도라는 지표 하나만 봐도 훌륭해 보이지만 실제로는 쓸모가 없다. → **Precision / Recall / F1** 은 정확도가 숨기는 진실을 드러내기 위한 지표.

혼동행렬: 네 개의 숫자

환자 100명 검사 → **혼동행렬**(confusion matrix): TP=18, FP=2, FN=7, TN=73.

실제 \ 예측	양성 예측	음성 예측
실제 양성	True Positive (TP)	False Negative (FN)
실제 음성	False Positive (FP)	True Negative (TN)

- 실제 양성 = $18 + 7 = 25$ 명, 모델이 양성 예측 = $18 + 2 = 20$ 명.
- **Precision** = $\frac{TP}{TP+FP}$: “양성이라고 예측한 것 중 진짜 양성 비율” — **거짓 경보**를 얼마나 피했는가.
- **Recall** = $\frac{TP}{TP+FN}$: “실제 양성 중 잡아낸 비율” — **놓친 양성**이 얼마나 적은가.
- **F1** = $\frac{2 \cdot P \cdot R}{P+R}$: P와 R의 **조화평균** — 둘 다 낮으면 F1도 낮아지도록, 한쪽만 높다고 후하게 주지 않음.

손으로 한 번: Precision · Recall · F1 계산

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{18}{18 + 2} = \frac{18}{20} = 0.9$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{18}{18 + 7} = \frac{18}{25} = 0.72$$

$$F1 = \frac{2 \times 0.9 \times 0.72}{0.9 + 0.72} = \frac{1.296}{1.62} = 0.8$$

- **Precision 0.9** = “양성이라고 예측한 20명 중 18명이 진짜 양성”.
- **Recall 0.72** = “실제 양성 25명 중 18명만 잡아냈다” (7명 놓침).

이 예에서 Precision > Recall 이면, 모델이 “확신이 설 때만 양성 판정”하는 쪽으로 **약간 보수적**이라는 신호 — 아래에서 다룰 **임계값 조정**이 바로 이 균형을 바꾸는 손잡이.

Precision-Recall 트레이드오프

- 임계값(threshold)을 0.5가 아니라 **0.9로 올리면** — Precision **오름** (확신 있을 때만 양성 판정) / Recall **떨어짐** (애매한 양성을 놓침).
- 임계값을 **낮추면** — Recall **오름** / Precision **떨어짐**.

PR-AUC

PR-AUC(Precision-Recall 곡선 아래 면적)는 이 트레이드오프 **전 구간**에서의 성능을 **임계값 하나에 의존하지 않고** 요약한 지표 — 특히 **클래스 불균형이 심할 때** accuracy보다 훨씬 신뢰할 수 있다.

스팸 필터: 임계값을 움직여보자 (설정)

같은 모델의 예측 확률을 임계값 0.5, 0.6, 0.1로 자를 때 혼동행렬이 어떻게 달라지는지 — “모델 성능”은 사실 **“임계값 + 모델”**의 성능.

- **스팸 필터** 두 특징: x_1 = 스팸 단어 빈도 (실제 메일에서 10~36개 범위), x_2 = “발신자 미등록” 여부 (0/1).
- 정규 메일은 스팸 단어 적고 대부분 발신자 등록.
- 스팸의 70%는 “분명한” 스팸, 30%는 정규 메일과 겹치는 **“교묘한” 스팸** — 임계값을 아무리 올려도 잡을 수 없는 스팸이 항상 남음.
- 전체 500통 중 스팸 100통 (20%), **테스트 세트 150통** (스팸 23통, 15.3%).

StandardScaler로 표준화 후 본문 logistic_gradient_descent로 학습 — 스케일링 필수: x_1 의 스케일(10~36)이 x_2 (0/1)의 30배라, 안 하면 2.1절 “학습률의 함정”이 그대로 살아남고 왜곡된 GD가 된다.

임계값 sweeping: 혼동행렬의 변화

임계값	TP	FP	FN	TN	정확도	Precision	Recall
0.05	21	119	2	8	0.193	0.150	0.913
0.10	19	62	4	65	0.560	0.235	0.826
0.20	16	24	7	103	0.793	0.400	0.696
0.30	12	14	11	113	0.833	0.462	0.522
0.40	11	7	12	120	0.873	0.611	0.478
0.50	10	2	13	125	0.900	0.833	0.435
0.60	9	0	14	127	0.907	1.000	0.391
0.80	4	0	19	127	0.873	1.000	0.174
0.95	2	0	21	127	0.860	1.000	0.087

(비용 $FP + 10 \cdot FN$ 였을 다음 프레임에서.)

1. Precision/Recall 트레이드오프가 그대로: 0.05 $\rightarrow$ 0.95 올리면 Precision 0.150 $\rightarrow$ 1.000, Recall 0.913 $\rightarrow$ 0.087 — “양성 판정 확신 문턱”을 높이면 오경보가 사라지지만 진짜 양성도 놓침.

비용 최소화 임계값은 0.5가 아니다

- **2. 비용 최소화 임계값은 0.5가 아니다.** FP(정상 메일을 스팸으로 오진 → 재검사 1건 비용)와 FN(스팸을 놓침 → 진단 지연, **비용 10배**) 합산:

$$\text{cost} = \text{FP} + 10 \cdot \text{FN}$$

0.05~0.95까지 0.01 간격 sweeping → **비용 최소 임계값** ≈ 0.14 (FN=4, FP=44, **비용 84**) — 0.5(비용 132)보다 **훨씬 낮**. “FN이 10배로 비싸다”는 비용 구조가 임계값을 0.14로 끌어내린 것.

- 임계값은 **비용 함수로 선택**하는 하이퍼파라미터.

3. 정확도는 오히려 떨어진다

임계값 0.5에서 정확도 **0.900**, 0.30에서 **0.833** — 0.5보다 **낮**다. 하지만 비용은 132 → **124**로 **떨어진다**.

“정확도가 더 낮은 모델이 더 쓸모있다”는 **역설** — 정확도는 “전체 중 맞힌 비율”을, 비용은 “특정 오류의 가중 합”을 재기 때문.

정확도 하나만 보고 임계값을 정하면, 실제로는 비용을 더 많이 내는 지점을 고르게 된다.

정리: PR-AUC의 기준선은 “무조건 0” 모델

클래스 불균형이 심할 때 PR-AUC의 **하한(baseline)**은 “무조건 음성으로 예측하는 모델”의 정확도 = **양성 비율**.

- 임계값을 **매우 낮추면** “모두 양성” → Recall=1.0, Precision=양성 비율 — PR 곡선의 y-좌표 (Precision)는 최대 양성 비율까지만 올라감.
- 임계값을 **매우 높이면** “모두 음성” → Precision=1.0 (양성을 예측한 게 없으므로), Recall=0 — PR 곡선은 (0,1)에서 시작.

핵심

PR-AUC는 항상 “양성 비율”과 1.0 사이의 값.

양성 비율 15%면 “무조건 0” 모델도 PR-AUC=0.15를 받는다.

검증: 모델 vs “무조건 0” (스팸 데이터)

실습 노트북 § 5 — 위 스팸 데이터 (테스트 세트 양성 비율 15.3%):

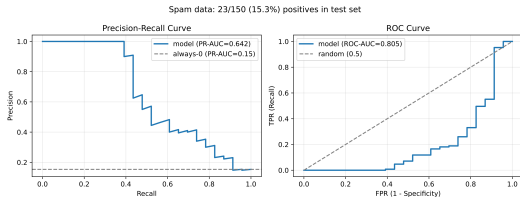
	정확도	PR-AUC / ROC-AUC
학습한 모델	0.847 (0.5)	0.642 / 0.805
무조건 0 모델	0.847	0.153 (= 양성 비율 0.15와 일치) / 0.5

결론

모델의 정확도(0.847 @ 0.5)와 “무조건 0”의 정확도(0.847)가 **정확히 같은데도** PR-AUC는 0.642 대 0.153 — **4배 차이**, ROC-AUC도 0.805 대 0.5.

정확도는 이 두 모델을 구분하지 못하지만, PR-AUC와 ROC-AUC는 구분한다 — 불균형 데이터에서 정확도가 “허위 지표”가 되는 이유.

PR 곡선과 ROC 곡선



스팸 데이터에서 임계값을 움직일 때의
Precision-Recall 곡선(좌)과 ROC 곡선(우) —
“무조건 0” 모델의 PR-AUC는 정확히 양성 비율(0.15),
ROC-AUC는 0.5.

- (좌) PR 곡선에서 **점선** = “무조건 0” 모델의 PR-AUC (= 양성 비율 0.15).
- (우) ROC 곡선에서 **점선** = 무작위 모델 (ROC-AUC=0.5).
- 모델 곡선이 점선보다 **위에 있는지** 보기 — 이 차이가 “모델이 양성의 **순서**를 얼마나 잘 짜는지”를 보여준다.

ROC-AUC vs PR-AUC: 언제 무엇을 보아야 하는가

ROC-AUC

- TPR-FPR 쌍을 임계값별로 그리는 곡선.
- **클래스 균형이 거의 맞을 때** 좋은 스크리닝 지표.
- 무작위 0.5, 완벽 1.0.
- **불균형이 무너지면** “무조건 0” 모델의 ROC-AUC가 여전히 0.5라, 0.15% 양성에서 ROC-AUC=0.51도 “무작위보다 낫다”로 읽혀 **과대평가**되기 쉬움.

PR-AUC

- **클래스 불균형이 심할 때** 더 “정직한” 지표 — 기준선이 양성 비율이라, 양성 15%에서 PR-AUC=0.151이면 “무조건 0과 거의 같은 모델”이 바로 읽힘.
- ROC-AUC=0.51은 “무작위보다 약간 낫다”로 읽혀 **절대 척도가 다름**.

실무 권장 흐름

ROC-AUC로 후보 모델 스크리닝 → PR-AUC와 비용 곡선으로 최종 임계값 선택.

ROC-AUC는 “순서 짓기 능력”을 빠르게 비교하고, PR-AUC + 비용 최소화는 “이 문제를 실제로 풀 때 어떤 임계값이 가장 이익”을 결정.

자주 하는 실수 (1): 스케일링 없이 경사하강법이 왜곡

- 선형회귀(2.1절)에서는 스케일링이 “학습률 선택의 **편의**” 였지만, 로지스틱회귀에서는 **더 심**.
- 시그모이드의 그래디언트 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ 가 $z = w^T x$ 에 **의존** — 특징 스케일이 30배로 다르면 z 의 크기 자체가 30배 차이.
- 스케일이 큰 특징 쪽 그래디언트가 **더 커짐** → 경사하강법이 “스케일 큰 특징을 **먼저 학습**”하는 왜곡된 경로를 타고, 학습률이 스케일에 맞춰 조정되지 않으면 **발산하거나 극도로 느려짐**.

실전 표준

StandardScaler 로 스케일링 후 학습 — 이 절의 실습 노트북도 스케일링 후.

자주 하는 실수 (2): 시그모이드 오버플로

$\text{sigmoid}(z)$ 를 직접 계산하면 **오버플로**가 날 수 있다.

- $z = -1000$ 이면 $e^{-z} = e^{1000}$ 이 float64 범위를 넘어 $\text{inf} \rightarrow 1/(1 + \text{inf}) = 0$ — **실제로 0이 아니라 “0과 거의 같은 값”을 0으로 잘림.**
- $z = +1000$ 이면 $e^{-z} = e^{-1000} \approx 0 \rightarrow 1/(1 + 0) = 1$ — “1과 거의 같은 값”을 1로 잘림.
- 둘 다 확률 0 또는 1에 가까울 때 **손실 $-\log p$ 가 $\log(0) = -\text{inf}$ 가 되어 NaN.**

실전 해결책 (택 1):

- `scipy.special.expit(z)` (C로 구현된, 오버플로 없는 시그모이드)
- `np.where(z >= 0, 1/(1+np.exp(-z)), np.exp(z)/(1+np.exp(z)))` — 음수/양수 영역 분산 계산
- 손실 계산 시 `np.clip(h, 1e-12, 1-1e-12)` 로 0/1에 딱 붙지 않게 클리핑

자주 하는 실수 (3): 임계값 0.5를 “모델 성능”으로 보기

- “정확도가 왜 75%야?”라고 물었을 때, 답은 “모델이 나쁘다”가 아니라 **“임계값 0.5가 이 문제의 비용 구조에 맞지 않는다”**일 수 있다.
- 2.1절 선형회귀의 “예측값”이 숫자 **자체**가 최종 답인데, 로지스틱회귀의 “예측 확률”은 **임계값을 적용하기 전의 중간 값**.

정확한 순서

“모델이 나쁘다”고 하기 전에:

- ① **PR-AUC / ROC-AUC** 로 순위 능력을 **먼저** 확인,
- ② **비용 구조에 맞는 임계값**을 sweeping해서 “이 임계값에서 나쁘다”를 확인,
- ③ 그때 “모델이 나쁘다”고 결론.

순서가 반 거꾸면 “모델을 갈아엎어야 하나”에서 “임계값을 0.14로 바꾸면 된다”로 바뀌지 않는다.

실습: sklearn으로 평가 지표 한 번에

```
from sklearn.metrics import classification_report, average_precision_score

y_true = [1, 1, 1, 0, 0, 0, 1, 0]
y_pred = [1, 0, 1, 0, 1, 0, 1, 0]
y_scores = [0.9, 0.4, 0.8, 0.3, 0.6, 0.2, 0.7, 0.1] # 임계값 0.5 적용전확률

print(classification_report(y_true, y_pred, target_names=["정상", "스팸"]))
print("PR-AUC:", round(average_precision_score(y_true, y_scores), 3))
```

- classification_report: P/R/F1을 **클래스별 한 번에** 출력.
- average_precision_score: 임계값을 **하나로 고정하지 않고** y_scores(확률) 전체를 보고 PR-AUC 계산 — **순위(ranking) 능력**을 평가.

같은 예측에 두 종류의 PR-AUC

	PR-AUC
y_scores (임계값 스윙)	0.95
y_pred (임계값 0.5 고정)	0.688

핵심

같은 데이터, 같은 모델인데 임계값을 “고정”했느냐 “스윙”했느냐에 따라 지표가 **0.26 차이**.
임계값 하나에 **강한** 지표 (임계값 0.5 성능만) 와 모델의 **순서짓기 능력**을 평가하는 지표의 차이가 수치로 보이는 것. (실습 노트북 § 6에서 재확인)

실습 노트북 지도 + 파이프라인 요약

- § 1 스팸 데이터 만들고 스케일링
- § 2 본문 logistic_gradient_descent를 numpy로 학습 (그래디언트 $(h - y) \cdot x$ 확인)
- § 3 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ 를 수치로 검증
- § 4 임계값 sweeping → P/R 트레이드오프와 비용 최소화 임계값 (≈ 0.14) 직접 계산
- § 5 PR-AUC/ROC-AUC와 “무조건 0” 모델의 기준선 (= 양성 비율) 검증
- § 6 본문 실습 코드 실행

파이프라인 요약

선형회귀와 로지스틱회귀는 겉보기엔 다른 문제(숫자 예측 vs 확률 예측)를 풀지만, 뜯어보면 “**선형모델** → **손실함수 정의** → **경사하강법으로 최소화**”라는 **완전히 같은 파이프라인** — 이 패턴은 **이번 학기 내내 반복된다**.

자주 묻는 질문 (1): F1은 왜 조화평균인가

Q. F1은 Precision과 Recall을 그냥 평균 내지 않고 조화평균을 쓰는 이유는?

- **산술평균**은 한쪽이 극단적으로 낮아도 다른 쪽이 높으면 값이 크게 낮아지지 않음 — 예: Precision=1.0, Recall=0.01이면 산술평균 0.505로 “꽤 높아 보이지만” 사실상 쓸모없는 모델 (양성을 거의 못 잡아냄).
- **조화평균**은 **작은 값 쪽에 훨씬 민감** — 같은 경우 $F1 = 2 \times 1.0 \times 0.01 / (1.0 + 0.01) \approx \mathbf{0.0198}$ 으로 훨씬 낮게 나와 “둘 중 하나라도 극단적으로 나쁘면 나쁜 모델”이라는 판단을 더 정확히 반영.

자주 묻는 질문 (2): 임계값 0.5는 항상 최선인가

아니다 — 임계값은 문제의 성격에 맞춰 조정하는 **하이퍼파라미터**.

암 검진: 양성 놓침 (FN) 비용 $\gg$ 오진 (FP) 비용
→ 임계값을 0.5보다 **낮춰서 Recall 올리는 쪽**.

스팸 필터: 정상 메일 오거름 (FP) 비용이 더 부담
→ 임계값을 **높여서 Precision 올리는 쪽**.

PR-AUC가 유용한 이유

아직 임계값을 **정하기 전에**, 모델 자체의 **순위 능력**을 먼저 평가할 수 있기 때문.

자주 묻는 질문 (3): 불균형에서 무엇을 봐야 하나

Q. 클래스 불균형이 심한 데이터에서 PR-AUC와 ROC-AUC 중 무엇을 봐야 하나? → **PR-AUC** 우선.

- 양성 15% 데이터: “무조건 0” 모델의 **PR-AUC = 0.15**, **ROC-AUC = 0.5**.
- 학습한 모델의 **PR-AUC = 0.16**이면 “거의 무조건 0과 같은 모델”이 **바로 읽힘**.
- 반면 **ROC-AUC = 0.51**은 “무작위보다 약간 낫다”로 해석되어 **과대평가**되기 쉬움.
- 불균형이 심할수록 **PR-AUC의 기준선(양성 비율)**이 더 “정직”.

ROC-AUC는 스크리닝(후보 모델 비교)에는 여전히 유용하지만, **최종 선택은 PR-AUC + 비용 곡선**으로.

자주 묻는 질문 (4): 왜 그래디언트가 같은 형태인가

Q. 교차 엔트로피의 그래디언트가 선형회귀와 같은 형태 $(h - y) \cdot x$ 인 이유는?

-
- (1) $\frac{\partial J}{\partial h} = -\frac{y}{h} + \frac{1-y}{1-h}$
 - (2) $\frac{\partial h}{\partial z} = \sigma(z)(1 - \sigma(z)) = h(1 - h)$
 - (3) 세 조각을 곱하면 $h(1 - h)$ 가 **통째로 약분**되어 $(h - y)$ 만 남음
-

해석

“**시그모이드의 미분이 정확히 시그모이드의 출입구**”라서, 손실함수의 “벌”과 시그모이드의 “압축”이 **상쇄**되는 것.

이 약분이 없으면 로지스틱회귀의 경사하강법은 선형회귀와 **다른 코드**가 되어야 하는데, 실제로는 **코드 구조가 완전히 동일** — 이 절의 “같은 패턴” 강조의 이유.

확인 문제: 4문단

- ① $w^T x = -2$ 일 때 $h_w(x)$ 는 몇이고, 이 예는 음성으로 예측되는가? $w^T x = 3$ 일 때는? (힌트: $\sigma(z)$ 를 대입하고 0.5와 비교.)
- ② “무조건 음성으로 예측하는 모델”의 ROC-AUC는 항상 0.5인가? (힌트: 임계값을 움직여 TPR-FPR 쌍을 그려라.)
- ③ Precision=0.5, Recall=0.9 일 때 F1은 몇인가? (힌트: 조화평균 공식. 단답.)
- ④ “양성이 5%인 데이터에서 PR-AUC=0.051이면 이 모델은 쓸모가 있다”는 진술은 옳은가? (힌트: PR-AUC의 기준선이 양성 비율임을 떠올려라.)

이번 주차 정리

- **2.1 세 부분 빼대**(모델, 비용함수, 최적화)를 선형회귀에 조립 — 평균제곱오차가 볼록 → 지역 최소값 없음, $\alpha^* = 2/\lambda_{\max}$ 경계로 발산 정확히 계산, diabetes 실데이터로 검증.
- **2.2 정규방정식** $w^* = (X^T X)^{-1} X^T y$ — 미분 0으로 닫힌 해, **사영(수직성)** 해석, 조건수가 걸음 수를 정하고 **스케일링은 걸음 수만 바꾼다** (25,800배 차이), Gauss-Markov(“선형 무편차 안에서 최선”), **특이 행렬과 pinv**(최소범위 해).
- **2.3 로지스틱회귀** — 시그모이드를 **log-odds**로 읽는 법, **교차 엔트로피** (Shannon 정보이론), 그래디언트가 $(h - y) \cdot x$ 로 **약분**되어 코드 재사용, “정확도의 함정” → 혼동행렬 · P/R/F1 · 임계값 sweeping(비용 최소화 ≈ 0.14) · **PR-AUC**(기준선 = 양성 비율) · ROC-AUC 스크리닝 → PR-AUC로 최종.

다음 주 — Week 3. 생성 모델 관점의 분류: 나이브베이지스와 GDA

이번 강의 **판별(discriminative)** 방식(데이터에서 클래스 확률을 직접 추정) 대신, “데이터가 어떻게 만들어졌는지”를 먼저 모델링하고 **베이지 정리로 뒤집어** 확률을 구하는 **생성(generative)** 방식. 이 관점의 구분이 **학기 전체**에서 분류 방법들을 나눌 기준이 된다.